

Processos, Threads, Escalonamento e Inferência Local com Ollama

Leticia Cavalcanti — AV1 Sistemas Operacionais — Trilha A (Ollama + Open WebUI)

1. Trilha e modelo

- **Trilha A** — Ollama + Open WebUI (chat local)
- **Modelo:** Llama-3.2-3B-Instruct — família Llama 3.2, Instruct, ~3,21B parâmetros
- **Formato/quantização:** GGUF, Q4_K_M
- **Licença:** Llama 3.2 Community License
- Por quê: porte reduzido (dentro do limite de 10B), execução viável em CPU, facilita observar processos/threads/syscalls sem exigir GPU

2. Ambiente experimental

- Host: **Windows 11** + **Docker Desktop** (backend **WSL2**)
- Ambiente Linux observado: **contêineres** (`ollama` , `open-webui`) — Ubuntu 24.04, kernel `6.6.87-microsoft-standard-WSL2`
- CPU: Intel i5-10210U, **4 CPUs** alocadas, **sem GPU** (execução 100% CPU)
- RAM: ~5,8 GiB alocados ao Docker Desktop
- Ollama 0.34.0 · modelo `llama3.2:3b` (GGUF Q4_K_M, 2,0 GB em disco)

3. Arquitetura

Navegador → :3000 Open WebUI → API HTTP local → :11434 Ollama → runner llama.cpp

- Comunicação via **API HTTP local** (`OLLAMA_BASE_URL`)
- Volumes Docker nomeados para persistência (modelos, conversas)
- Isolamento via **namespaces/cgroups** do container

4. Processos e threads observados

- **Repouso:** `ollama serve` (PID 1), 25 threads
- **Durante inferência:** processo filho `llama-server` (PID 149, PPID 1), 9 threads, ~91,5% CPU
- `ollama` = processo supervisor/API; trabalho pesado delegado a processo filho sob demanda
- Uso de memória do container `ollama` : até **3,7 GiB** (contexto grande)

5. Chamadas de sistema (strace -f -c -p 1)

- 6.644 chamadas capturadas durante uma requisição controlada
- `futex` = **72,8% do tempo** → sincronização entre threads de inferência
- `epoll_pwait`, `socket`, `accept4` → servidor HTTP local
- `read`, `openat`, `pread64` → leitura dos pesos do modelo
- `madvise` → gerência de memória do KV-cache

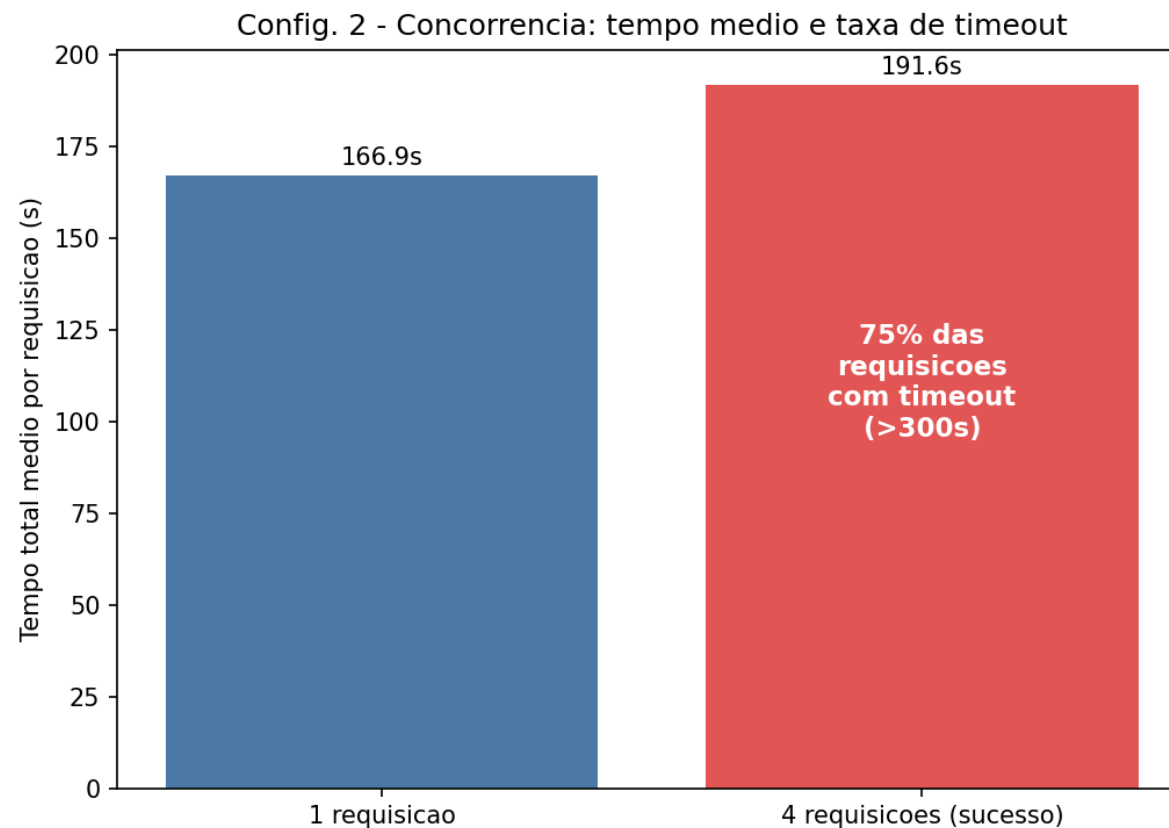
6. Metodologia dos experimentos

12 rounds, 3 configurações, prompt curto x longo, ≥ 2 repetições:

1. **Padrão** — sequencial, `num_ctx` default
2. **Concorrência** — 1 vs 4 requisições simultâneas
3. **Contexto** — `num_ctx` 2048 vs 8192

Métricas via API do Ollama (`total_duration`, `eval_count` ...) + `docker stats` + `ps -eo nlwp`

7. Resultado principal — concorrência



Concorrência não melhorou vazão — piorou. CPU (4 núcleos, sem GPU) satura, 75% das requisições não completam em 300s.

8. Resultado — contexto

num_ctx	Tempo médio (s)	Memória
2048	192,2	~2,75–2,82 GiB
8192	232,9 (+21%)	~3,58–3,68 GiB (+30%)

- Troca de `num_ctx` força **recarregamento do modelo** (~6,3–6,7s extra na 1ª requisição)

9. Limites e riscos

- Execução 100% CPU — sem comparação CPU/GPU real
- Virtualização adicional (Docker Desktop/WSL2) pode introduzir ruído nos tempos absolutos
- Amostra mínima (12 execuções) — carga do host Windows pode ter afetado medições
- Timeouts em concorrência=4 limitaram o N de amostras bem-sucedidas nesse cenário

10. Decisões adotadas

- Uso de **containers** (Docker Desktop/WSL2) como "ambiente Linux" — válido pela Trilha A
- Artefato GGUF via biblioteca oficial do Ollama (`llama3.2:3b`), não do repositório *gated* do HF — mesmo modelo-base
- `cap_add: SYS_PTRACE` adicionado para permitir `strace` dentro do container

Conclusão

Responsividade e uso de recursos de um sistema local de IA generativa dependem menos do tamanho do modelo (fixo, 3B) e mais da **configuração de execução** — concorrência e contexto — que interage diretamente com o escalonamento e paralelismo do SO subjacente.

Repositório: github.com/leticiaacavalcaanti/SO_UFS_2026_2_Cavalcanti_Leticia